

mindX observability stack — production-grade self-hosted blueprint

Audience: Gregory (codephreak / Professor Codephreak), senior dev, no hand-holding. All versions and licenses verified May 2026. Apache-2.0 sovereignty alignment is enforced; AGPL/BUSL/SSPL/CC-BY-NC components are flagged with mitigation paths.

The recommendation in one sentence: build mindX's observability as a **CNCF/Apache-2.0 spine** (Prometheus 3.11 + OTel Collector 0.151 + Jaeger v2 + Vector + Thanos) wrapped around a **Langfuse v3 (MIT) LLM-observability pane** instrumented by **OpenLLMetry (Apache-2.0)** emitting **OpenInference + OTel GenAI semconv** spans, with an **eval pipeline** (Inspect AI + lm-eval-harness + DeepEval + Ragas) that pushes per-checkpoint outcome metrics to **Pushgateway** so every aGLM tier promotion is gated on demonstrable benchmark improvement and surfaced in Grafana alongside openBDK's bridge health on a single shared cross-project SLO dashboard. Grafana, Loki, and Tempo are accepted at arm's length under AGPL-3.0 (used unmodified, separated by network boundary); Jaeger v2 + Vector replace Tempo/Promtail/Alloy in the sovereign path; SeaweedFS replaces MinIO (which entered maintenance mode and is being archived in 2026); OpenBao replaces Vault (BUSL); ClearML-server (SSPL) and Patronus Lynx weights (CC-BY-NC) are excluded from production. The structural insight that organizes the whole design: **outcome-quality benchmarking is a first-class telemetry signal** — every aGLM checkpoint's eval scores live in the same Prometheus TSDB as latency and GPU utilization, sharing labels (`checkpoint_id`, `tier`, `aglm_release`) so that drift, regression, and Goodhart-gaming all become PromQL queries and Alertmanager rules rather than ad-hoc reports.

1. The metrics backbone shared with openBDK

The shared spine is **Prometheus 3.11.1** (Apache-2.0, <https://github.com/prometheus/prometheus>) deployed once per project (`prom-openbdk`, `prom-mindx`) with a **Thanos sidecar** (Apache-2.0, <https://github.com/thanos-io/thanos>, v0.40.x) on each, pointing to a shared **SeaweedFS** (Apache-2.0, <https://github.com/seaweedfs/seaweedfs>) S3-compatible bucket. A single **Thanos Querier** fans out across both sidecars, providing the global view without commingling raw TSDBs — critical for the "shared dashboards but sovereign data" goal. Reject **federation** via `/federate` for anything other than pre-recorded aggregates (high cardinality kills it); reject **Mimir** (AGPL-3.0) and **MinIO** (AGPL-3.0 + entered maintenance mode December 2025, archive April 2026 → Web UI moved to commercial AISTor at ~\$96k/yr; do not build new infra on it). **VictoriaMetrics** (Apache-2.0) is the equally-valid alternative if you want a single-binary long-term store with ~5× less RAM than Mimir, but Thanos's sidecar pattern integrates more cleanly with two pre-existing Prometheus instances.

[My blog](#)

[My blog](#)

Retention on each Prom is set to 15 days local (`--storage.tsdb.retention.time=15d`); long-term lives in Thanos. Native OTLP ingestion is on (`--enable-feature=otlp-write-receiver`) so the OTel Collector can also (`prometheusremotewrite`) directly. Remote_write tuning at (`queue_config.max_samples_per_send=10000`), (`capacity=20000`), (`min_shards=4`), (`max_shards=200`) is the working baseline for billion-sample/day workloads. Prometheus 3.11's distroless image (`prom/prometheus:v3.11.1-distroless`, UID 65532) is the production target.

[GitHub](#)

`prometheus.yml` for the mindX node — copy-pasteable:

yaml

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    cluster: mindx
    project: mindx
    region: pythai-net

rule_files:
- /etc/prometheus/rules/recording_rules.yml
- /etc/prometheus/rules/alert_rules.yml
- /etc/prometheus/rules/eval_drift_rules.yml

alerting:
  alertmanagers:
    - static_configs: [{targets: ['alertmanager:9093']}]]

scrape_configs:
- job_name: prometheus
  static_configs: [{targets: ['localhost:9090']}]]

- job_name: mindx_api
  metrics_path: /metrics
  scheme: https
  static_configs: [{targets: ['mindx.pythai.net']}]]
  tls_config: {insecure_skip_verify: false}
  basic_auth:
    username: prom
    password_file: /etc/prometheus/secrets/mindx_api_scrape_pw

- job_name: mindx_blackbox_http
  metrics_path: /probe
  params: {module: [http_2xx_mtls]}
  static_configs:
    - targets:
      - https://mindx.pythai.net/healthz
      - https://mindx.pythai.net/v1/agents/ping
      - https://agenticplace.pythai.net/healthz
      - https://bankon.pythai.net/healthz

relabel_configs:
- {source_labels: [__address__], target_label: __param_target}
- {source_labels: [__param_target], target_label: instance}
- {target_label: __address__, replacement: blackbox-exporter:9115}
```

```
- job_name: node
  static_configs: [{targets: ['node-exporter:9100']}]}

- job_name: dcgm
  static_configs: [{targets: ['dcgm-exporter:9400']}]}

- job_name: podman
  static_configs: [{targets: ['prometheus-podman-exporter:9882']}]}

- job_name: process
  static_configs: [{targets: ['process-exporter:9256']}]}

- job_name: nats
  static_configs: [{targets: ['nats-exporter:7777']}]}

- job_name: qdrant
  metrics_path: /metrics
  static_configs: [{targets: ['qdrant:6333']}]}

- job_name: meilisearch
  metrics_path: /metrics
  bearer_token_file: /etc/prometheus/secrets/meili_metrics_token
  static_configs: [{targets: ['meilisearch:7700']}]}

- job_name: kuzu_exporter
  static_configs: [{targets: ['kuzu-exporter:9354']}]}

- job_name: pushgateway_evals
  honor_labels: true
  static_configs: [{targets: ['pushgateway:9091']}]}

- job_name: otel_collector_internal
  static_configs: [{targets: ['otel-collector:8888']}]}

- job_name: langfuse
  metrics_path: /api/public/metrics
  static_configs: [{targets: ['langfuse-web:3000']}]}

remote_write:
- url: http://thanos-receive:19291/api/v1/receive
  queue_config: {max_samples_per_send: 10000, capacity: 20000, min_shards: 4, max_
```

Grafana 13.0.1 (AGPL-3.0 since 2021 — *unmodified self-hosting is unrestricted*; the AGPL only triggers source-disclosure if you modify and host as a service) provides one Org with two folder-scoped RBAC trees (`/openbdk/*`), (`/mindx/*`) plus a third (`/shared/*`) for the cross-project SLO dashboard. Grafana 13's **Git Sync GA** is the canonical dashboards-as-code path: dashboards live in `mindx_observability/grafana/dashboards/*.json`, two-way reconciled to a Git repo. **Service accounts replace API keys** post-v9. OIDC against **Keycloak 26.6** (Apache-2.0) gives `[auth.generic_oauth]` with `role_attribute_path` JMESPath like `contains(groups[*], 'mindx-admin') && 'Admin' || contains(groups[*], 'mindx-editor') && 'Editor' || 'Viewer'`. SAML is Enterprise-only; OIDC is OSS. The **shared dashboard library** lives at `mindx_observability/grafana/dashboards/shared/`: `cross_project_slo.json` (probe success, error budget burn, p95 latency, hallucination rate, eval-score sparklines per tier — single pane covering both openBDK bridge health and mindX agent health) and `aglm_outcome_quality.json` (the eval-score-over-time view). (Grafana + 5)

Exporters on the mindX node, all Apache-2.0: **node_exporter v1.11.1** at `:9100`; **NVIDIA DCGM exporter 4.5.3-4.8.2-distroleless** at `:9400` with custom counters CSV enabling `DCGM_FI_PROF_SM_OCCUPANCY`, `DCGM_FI_PROF_PIPE_TENSOR_ACTIVE`, `DCGM_FI_DEV_NVLINK_CRC_FLIT_ERROR_COUNT_TOTAL`, `DCGM_FI_DEV_XID_ERRORS`, `DCGM_FI_DEV_ECC_DBE_VOL_TOTAL`, plus the standard utilization/VRAM/power set; **prometheus-podman-exporter** (the official replacement for cAdvisor in Podman environments — Podman does **not** expose `/metrics` natively despite occasional claims) at `:9882`; **blackbox_exporter v0.27.0** at `:9115` for HTTP probes against `mindx.pythai.net`, `agenticplace.pythai.net`, `bankon.pythai.net`; **process_exporter v0.8.7** at `:9256`. For per-training-run GPU attribution outside Kubernetes, use DCGM's `--hpc-job-mapping-dir` flag pointed at `/var/run/dcgm-jobs/` where each file is named after the GPU UUID and contains the `automindx_run_id`. (GitHub + 7)

Alertmanager v0.32.0 (Apache-2.0) handles routing with `group_by: [alertname, cluster, service, tier]`, `group_wait: 30s`, `group_interval: 5m`, `repeat_interval: 4h`, HA via gossip on `:9094`. **Karma** (Apache-2.0, <https://github.com/primitive/karma>) provides the unified UI across openBDK + mindX alerts with silence management. SLO-style alerts: probe down for `mindx.pythai.net` >2m → page; aGLM eval regression >3% on any flagship benchmark → ticket; `kc_projection_lag_seconds > 300` → page; `mindx_hallucination_rate_5m > 0.05` → page; `DCGM_FI_DEV_XID_ERRORS > 0` → page (irrecoverable GPU error); `rate(jetstream_consumer_num_redelivered[5m]) > 1` for 10m → poison-message ticket. (GitHub + 3)

2. The OTel tracing layer

OpenTelemetry Collector contrib v0.151.0 (Apache-2.0) is the central pipeline, deployed twice — agent on each host (resource detection, host metrics, fast handoff) and a **gateway cluster** that does tail sampling, PII redaction, and fan-out. Agents run as Quadlets; gateways as a 3-replica

systemd group behind a local DNS round-robin. **Tempo is rejected** in the sovereign path (AGPL-3.0); **Jaeger v2 with ClickHouse backend** (Apache-2.0, <https://github.com/jaegertracing/jaeger>, v2 GA Nov 2024, v1 EOL'd Dec 31 2025) is the trace store. Jaeger v2 is itself built on the OTel Collector framework, so the deployment story unifies. **Vector v0.55.0** (MPL-2.0, <https://github.com/vectordev/vector>) replaces both Promtail (EOL March 2 2026) and Alloy (AGPL-3.0) as the host log/metric agent — disk-buffered, VRL-capable for in-flight redaction, native OTLP source/sink and Loki sink. [Jaeger + 4](#)

`otel_collector_config.yaml` — gateway profile, copy-pasteable:

yaml

```

receivers:
  otlp:
    protocols:
      grpc: {endpoint: 0.0.0.0:4317}
      http: {endpoint: 0.0.0.0:4318}
  prometheus/self:
    config:
      scrape_configs:
        - job_name: otelcol
          scrape_interval: 30s
          static_configs: [{targets: ['localhost:8888']}]}

processors:
  memory_limiter:
    check_interval: 1s
    limit_percentage: 80
    spike_limit_percentage: 25
  resourcedetection:
    detectors: [env, system, process]
  transform/genai_redact:
    error_mode: ignore
    log_statements:
      - context: log
        statements:
          - replace_pattern(body, "(?i)(api[_-]?key|secret|token|password)\\s*[:=]\\s*", "[REDACTED]")
    trace_statements:
      - context: span
        statements:
          - set(attributes["gen_ai.prompt.0.content"], "[REDACTED]") where attribute_type == string

tail_sampling:
  decision_wait: 30s
  num_traces: 100000
  policies:
    - {name: errors, type: status_code, status_code: {status_codes: [ERROR]}}
    - {name: slow, type: latency, latency: {threshold_ms: 2000}}
    - {name: aglm_eval, type: string_attribute, string_attribute: {key: aglm.eval, value: "aglm.eval"}}
    - {name: sample_5pct, type: probabilistic, probabilistic: {sampling_percentage: 5}}

batch:
  send_batch_size: 8192
  timeout: 5s

attributes/mindx:
  actions:
    - {key: project, value: mindx, action: upsert}

```

```

exporters:
  otlp/jaeger:
    endpoint: jaeger-collector:4317
    tls: {insecure: true}
  prometheusremotewrite:
    endpoint: http://prometheus:9090/api/v1/write
    resource_to_telemetry_conversion: {enabled: true}
  otlphttp/langfuse:
    endpoint: http://langfuse-web:3000/api/public/otel
    headers:
      Authorization: "Basic ${env:LANGFUSE_BASIC_AUTH}"
  loki:
    endpoint: http://loki:3100/loki/api/v1/push
    default_labels_enabled: {exporter: false, job: true}

extensions:
  health_check: {endpoint: 0.0.0.0:13133}
  pprof: {endpoint: 0.0.0.0:1777}
  zpages: {endpoint: 0.0.0.0:55679}

service:
  extensions: [health_check, pprof, zpages]
  telemetry:
    metrics: {address: 0.0.0.0:8888}
    logs: {level: info}
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, resourcedetection, transform/genai_redact, tail_sampling]
      exporters: [otlp/jaeger, otlphttp/langfuse]
    metrics:
      receivers: [otlp, prometheus/self]
      processors: [memory_limiter, resourcedetection, attributes/mindx, batch]
      exporters: [prometheusremotewrite]
    logs:
      receivers: [otlp]
      processors: [memory_limiter, transform/genai_redact, batch]
      exporters: [loki]

```

opentelemetry-python 1.41.1 (Apache-2.0) is wired into the FastAPI app via

`FastAPIInstrumentor.instrument_app(app)`, `HTTPXClientInstrumentor().instrument()`,
 and an explicit asyncio note: `run_in_executor` and FastAPI `BackgroundTasks` lose context

unless you `context.attach()` manually — for any agent step that hops to a thread pool, capture the active span and re-attach. Streaming responses must emit per-chunk events, never wrap the whole iterator in a single span. **Loki 3.7.1** is accepted at arm's length under AGPL-3.0 (we ship no modifications to Loki, only configs and dashboards — derivative-works clause is not triggered); if absolute Apache-2.0 purity matters, swap Loki for **OpenObserve** (Apache-2.0) or send logs to ClickHouse via Vector and skip the Loki tier entirely. Trace-log correlation via `trace_id`/`span_id` injection in the Python `LoggingInstrumentor` is mandatory; Grafana's "Logs to Trace" derived field rule on `^.*trace_id=([0-9a-f]+).*` makes click-through navigation work. `PyPI + 4`

3. LLM-specific observability — the verdict

Adopt Langfuse v3.172 self-hosted (MIT) as the primary LLM observability backend, instrument with OpenLLMetry (Apache-2.0), supplement OpenInference span kinds where richer agent taxonomy is needed. This combination was verified against the upstream state in May 2026 and stands on three legs: Langfuse v3 ingests OTLP natively at `/api/public/otel/v1/traces` (since v3.22.0) and reads `gen_ai.*` semconv attributes plus its own `langfuse.*` overrides, with no app-side Langfuse SDK code required if pure OTLP is used; the Langfuse OSS core is MIT (more permissive than Apache-2.0) with all tracing, prompt management, dataset, eval, session, and agent-graph features in-scope (only org-scoped advanced RBAC, SAML, audit logs, SCIM are EE-gated, which a sovereign single-team deployment does not need); ClickHouse acquired Langfuse late 2025 with a public commitment to keep it OSS. The operational footprint is heavy — Postgres + ClickHouse ≥ 24.3 (pin to 25.5.2 or 25.8, **avoid 25.6.x** memory regression on deletes; run all components in UTC) + Redis/Valkey + S3-compatible blob + two Node containers — but for a billion-row autonomous system this is appropriate. Set `LANGFUSE_RELEASE=<aglm_checkpoint_hash>` so every trace is keyed to a checkpoint, enabling per-checkpoint regression analytics from the same data plane that drives Prometheus metrics. `Langfuse + 9`

Reject Helicone (entered maintenance mode after Mintlify acquisition March 2026; proxy-shaped architecture wrong for in-process agent tracing). **Reject Lunary** (main repository was abruptly deleted from GitHub in December 2025 — sovereignty risk regardless of code mirrors). **Phoenix** (Elastic 2.0, not OSI but self-hosting fully allowed and no feature gates) is the strong second choice — single-container deployment, best-in-class RAG retrieval inspection, the most OTel-native tool of any reviewed (it co-develops the OpenInference semconv) — recommended as a developer-laptop dev tool reading from local SQLite while production uses Langfuse. **OpenLIT v1.15.x** (Apache-2.0) is the dark-horse alternative for absolutist Apache-2.0 stacks: 3-component deployment (app + ClickHouse + bundled OTel Collector), strongly OTel-native, bundles GPU monitoring, smaller community than Langfuse but actively developed. `Helicone + 9`

`OTEL_SEMCONV_STABILITY_OPT_IN=gen_ai_latest_experimental` is set globally in pod env to opt into v1.40 GenAI semconv; the underlying spec is still Development status as of May 2026 (no Stable marker yet) — pin the semconv package version, treat the schema as movable, and use `gen_ai.provider.name` (not the deprecated `gen_ai.system`). Capture full prompt/completion content as **events** (`gen_ai.client.inference.operation.details`), not span attributes, gated by `OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT=false` by default with per-route override to `true` when content capture is needed; PII redaction lives in the Collector's `transform/genai_redact` processor on a `mindx.pii_class` attribute set by an in-app classifier. Token cost is computed at query time in Grafana via a recording rule keyed off `gen_ai.request.model` and a per-model price table maintained in `prometheus/recording_rules.yml`; latency p50/p95/p99 per model and per checkpoint comes from the `gen_ai.client.operation.duration` histogram with `histogram_quantile(0.95, sum by (le, gen_ai_request_model, aglm_checkpoint_id) (rate(gen_ai_client_operation_duration_bucket[5m])))`. `OpenTelemetry + 5`

The LLM observability comparison matrix:

Tool	Version (Q2 2026)	License	OTel-native	Self-host complexity	Cost	Extra
Langfuse	v3.172 / SDK v4.5	MIT core; commercial EE for advanced admin	Yes (OTLP/HTTP)	High (Postgres+ClickHouse+Redis+S3)	Yes	Yes (L d ex
Phoenix	v14.15	Elastic-2.0 (not OSI; self-host fully allowed)	Strongest	Low (single container/pip)	Yes	Yes tr
Helicone	maint. mode	Apache-2.0 (gateway is GPL-3.0)	Partial	Low	Yes	Li
OpenLLMetry	v0.60.0	Apache-2.0	It is OTel; v1.40 semconv	N/A (SDK)	Via attrs	N
Lunary	v1.9	Apache-2.0	Partial	Low	Yes	Yes
OpenLIT	v1.15	Apache-2.0	Strong	Low/Med (3 comp)	Yes	Yes
TruLens	2.7.2	MIT	Partial (consumes OTel)	Low	Limited	Bo
Laminar	active	Apache-2.0	Yes	Med	Yes	Yes
MLflow	3.12	Apache-2.0	Yes	Low	Yes	Yes

4. The outcome-quality benchmarking layer — the self-optimization assessment core

This is where the deliverable departs from a normal observability stack. mindX's claim to autonomous self-improvement is meaningless without a regression-proof eval pipeline that runs on every aGLM checkpoint and stores results in the same time-series store as the rest of the telemetry. The pipeline has three harness layers feeding one exporter: **lm-evaluation-harness v0.4.9.2** (MIT, EleutherAI, <https://github.com/EleutherAI/lm-evaluation-harness>) for academic-comparable deterministic scores (MMLU-Pro, GSM8K, HumanEval+, IFEval, BBH, GPQA, MATH-500, HellaSwag, ARC, WinoGrande) running over vLLM with `--batch_size auto`; **Inspect AI** (MIT, UK AISI, https://github.com/UKGovernmentBEIS/inspect_ai, with the 130+-eval `inspect_evals` registry) for agentic/safety/frontier evals (τ^3 -bench, SWE-bench Pro, BFCL V4 Agentic, HLE, BBEH, OSWorld, ARC-AGI-3, Inspect's scheming category); **DeepEval 4.0** (Apache-2.0, <https://github.com/confident-ai/deepeval>) plus **Ragas** (Apache-2.0, <https://github.com/explodinggradients/ragas>, ~v0.3.x) for RAG/agent LLM-as-judge metrics — faithfulness, response_relevancy, context_precision, context_recall, NoiseSensitivity, ToolCallF1, AgentGoalAccuracy, plus G-Eval via DeepEval's `GEval` class (the canonical implementation of the EMNLP 2023 method). **Promptfoo** (MIT, <https://github.com/promptfoo/promptfoo>, acquired by OpenAI March 2026 but remains MIT) sits in CI on every PR for prompt-regression sweeps and red-team scans. `(ADS + 4)`

Per-tier canonical batteries — these are the recommended starting points for the five aGLM tiers, structured as smoke (≤ 10 min) / core (≤ 2 h) / extended (overnight) / safety overlay:

- **Edge tier** (≤ 3 B, on-device): smoke = GSM8K + IFEval + HumanEval+; core = MMLU + HellaSwag + ARC-Challenge + TruthfulQA-MC2 + PIQA + WinoGrande; safety = DeepEval Toxicity + Bias on a 200-prompt set.
- **Mid tier** (~7-13B): smoke = IFEval + GSM8K + HumanEval+ + MBPP+; core = MMLU-Pro + BBH + MATH-500 + GPQA Main + AlpacaEval 2 LC + Arena-Hard-Auto; RAG = Ragas faithfulness + answer_relevancy + context_precision/recall on internal corpus + MultiHop-RAG; tool = BFCL V3 simple/parallel/multiple via Inspect Evals. `(LLM Leaderboard)`
`(Awesome Agents)`
- **Flagship tier**: GPQA Diamond + MMLU-Pro + BBEH (replacement for saturated BBH) + MATH-500 + AIME 2025/2026 + HLE + ARC-AGI-2; long-context RULER + BabiLong + LongBench v2; Arena-Hard-Auto + AlpacaEval 2 LC + WildBench; full Ragas v2 + MultiHop-RAG + RGB; BFCL V3 + V4 Agentic; τ^3 -bench retail+airline+banking_knowledge + GAIA + OSWorld with held-out subsets; HELM safety subset + AIR-Bench.
`(LLM Leaderboard + 4)`
- **Specialist-code tier**: HumanEval+ and MBPP+ kept only as legacy baselines (saturated, contaminated); **LiveCodeBench is the primary contamination-resistant code-gen benchmark**; SWE-bench Verified for compatibility, **SWE-bench Pro as primary** (1,865 multi-language, top model Claude Opus 4.5 only 45.9%, GPT-5 High ~23% — actually

discriminative), SWE-bench-Live monthly multilingual, Multi-SWE-bench; BFCL V3+V4 full categories; Spider 2.0 + BIRD with annotation-error caveat; Terminal-Bench 2.0 via Inspect Harbor. (Morph + 5)

- **Specialist-think tier:** GPQA Diamond + HLE + ARC-AGI-2/3 (frontier <1% on ARC-AGI-3, adopted as a model-card standard) + BBEH + AIME 2025/2026 + MATH-500 + FrontierMath (private) + OlympicArena; long-horizon GraphWalks + BabiLong + MuSR; agentic τ^3 -bench full + OSWorld + WebArena (with held-out blind subset and reward-hack awareness); BFCL V4 Agentic full; SelfCheckGPT-NLI on 500 free-form generations; Inspect Evals scheming category (HarmActionsEval, blackmail-replacement); maintain a private 10% sample of every benchmark and alert on public/private divergence >3%.

(MarkTechPost)

The **Berkeley RDI April 2026 finding** — that all 8 major agent benchmarks (SWE-bench, WebArena, OSWorld, GAIA, Terminal-Bench, FieldWorkArena, CAR-bench, AgentBench) are reward-hackable to ~100% via gold-answer leakage or `eval()` injection in the grader — is non-negotiable: **every benchmark in mindX's pipeline must have a private held-out subset, the eval VM must be sandboxed (no internet, no `eval()` on agent-controlled strings), and top scores must be cross-checked against an independent leaderboard** (Epoch AI / BenchLM / Artificial Analysis). Patronus Lynx is excluded from production because its weights are CC-BY-NC-4.0 (non-commercial) — for hallucination detection use **Vectara HHEM-2.1-Open** (Apache-2.0 weights, ~600MB, runs CPU-only ~1.5s/2k tokens) plus SelfCheckGPT-NLI (MIT) as a second pass; CRAG dataset is also CC-BY-NC and excluded from commercial scoring. CRAG is replaced by RGB + MultiHop-RAG for production RAG eval. (Berkeley + 4)

`eval_runners/aglm_eval_pusher.py` — copy-pasteable Pushgateway pattern using

`pushadd_to_gateway` so failed runs don't overwrite last-known-good metrics: (Robust Perception)

```
python
```

```

from prometheus_client import CollectorRegistry, Gauge, pushadd_to_gateway, delete_from_gateway
import json, time, os

registry = CollectorRegistry()

eval_score = Gauge('mindx_eval_score', 'aGLM benchmark score (0..1 normalized)',
    ['benchmark', 'tier', 'checkpoint_id', 'aglm_release', 'metric', 'judge', 'split'])
eval_pass_at_k = Gauge('mindx_eval_pass_at_k', 'pass@k for code/agent benchmarks',
    ['benchmark', 'tier', 'checkpoint_id', 'k'], registry=registry)
eval_duration_seconds = Gauge('mindx_eval_duration_seconds', 'Wall-clock eval duration',
    ['benchmark', 'tier', 'checkpoint_id'], registry=registry)
eval_samples_total = Gauge('mindx_eval_samples_total', 'Number of samples evaluated',
    ['benchmark', 'tier', 'checkpoint_id'], registry=registry)
eval_cost_usd = Gauge('mindx_eval_cost_usd', 'Eval cost USD',
    ['benchmark', 'tier', 'checkpoint_id', 'judge'], registry=registry)
eval_last_success_ts = Gauge('mindx_eval_last_success_timestamp_seconds',
    'Unix timestamp of last successful eval run',
    ['benchmark', 'tier', 'checkpoint_id'], registry=registry)
eval_holdout_divergence = Gauge('mindx_eval_holdout_public_divergence',
    'public_score - holdout_score (Goodhart sentinel)',
    ['benchmark', 'tier', 'checkpoint_id'], registry=registry)

def push_eval_result(result_path: str, pushgateway_url: str = 'pushgateway:9091'):
    r = json.load(open(result_path))
    bench, tier, ckpt, release = r['benchmark'], r['tier'], r['checkpoint_id'], r['aglm_release']
    for metric_name, metric_value in r['metrics'].items():
        eval_score.labels(bench, tier, ckpt, release, metric_name,
            r.get('judge', 'none'), r.get('split', 'public')).set(metric_value)
    for k, v in r.get('pass_at_k', {}).items():
        eval_pass_at_k.labels(bench, tier, ckpt, k).set(v)
    if 'public_score' in r and 'holdout_score' in r:
        eval_holdout_divergence.labels(bench, tier, ckpt).set(r['public_score'] - r['holdout_score'])
    eval_duration_seconds.labels(bench, tier, ckpt).set(r['duration_seconds'])
    eval_samples_total.labels(bench, tier, ckpt).set(r['n_samples'])
    eval_cost_usd.labels(bench, tier, ckpt, r.get('judge', 'none')).set(r.get('cost_usd', 0))
    eval_last_success_ts.labels(bench, tier, ckpt).set(time.time())

    pushadd_to_gateway(pushgateway_url, job='mindx_aglm_eval',
        grouping_key={'tier': tier, 'checkpoint': ckpt, 'benchmark': bench},
        registry=registry)

```

```
if __name__ == '__main__':
    import sys; push_eval_result(sys.argv[1])
```

A Ragas runner that samples 5% of live RAG traffic via NATS subject `mindx.rag.eval.samples`, scores faithfulness + answer_relevancy + context_precision against a self-hosted Llama-3 judge for sovereignty (or gpt-4o-mini for higher correlation when allowed), and emits the same metric family with `judge=` label, gives continuous outcome-quality telemetry on production traffic with no ground truth needed (faithfulness is reference-free).

Drift detection PromQL — the alert that catches autonomous regressions:

```
yaml
- alert: AglmEvalRegression
  expr: |
    (
      max_over_time(mindx_eval_score{tier="flagship"}[7d])
      - mindx_eval_score{tier="flagship"}
    ) / max_over_time(mindx_eval_score{tier="flagship"}[7d]) > 0.03
  for: 30m
  labels: {severity: page, project: mindx}

- alert: AglmHoldoutDivergence
  expr: abs(mindx_eval_holdout_public_divergence) > 0.03
  for: 1h
  labels: {severity: ticket, kind: goodhart_canary}

- alert: AglmEvalStale
  expr: time() - mindx_eval_last_success_timestamp_seconds > 86400 * 7
  for: 1h
```

A/B and shadow deployment: **Traefik v3 mirroring** (MIT) routes 100% to the incumbent and mirrors 100% as shadow to the candidate (mirrors do not return responses to the client — zero risk), with weighted graduation 5%→25%→50%→100% on `mindx_hallucination_rate_5m`, `mindx_tool_call_success_ratio`, `histogram_quantile(0.95, ...)` latency, and `mindx_eval_score` regression checks. **LiteLLM Router** (MIT, <https://github.com/BerriAI/litellm>) is the LLM-native equivalent if you prefer an LLM proxy. **Langfuse self-hosted's offline experiment-comparison + dataset versioning** closes the loop: run the same eval set against both checkpoints, compare in Langfuse UI, gate via Promptfoo CI threshold check.

DEV Community + 2

The eval framework matrix:

Framework	Version	License	RAG	Agent	Coverage	Prom integ.	Ver
DeepEval	4.0.0	Apache-2.0	★★★★★	★★★★★	50+	Easy	Prime
Ragas	0.3.x	Apache-2.0	★★★★★	★★★★	-	Easy	Prime
Promptfoo	0.95	MIT	★★★★	★★★	★★★★	CLI→JSON	CI/red
OpenAI Evals	maint.	MIT	★★	★★	★★★★	Med	Av sir
Inspect AI	May 2026	MIT	★★★★	★★★★★★	★★★★★★ (200+)	Easy	Prime flag
lm-eval-harness	0.4.9.2	MIT	★★	★★	★★★★★★	Easy	Re
HELM	0.5.14 (maint. Jun 2026)	Apache-2.0	★★	★★	★★★★★	Med	Leg
MTEB	Apr 2026	Apache-2.0	★★ retrieval	-	embeddings	Easy	Ma shi em
TruLens	2.7.2	MIT	★★★★★	★★★★★ OTel	-	Easiest	OT
OpenCompass	0.5.2	Apache-2.0	★★	★★	★★★★★★ (70+)	Med	Op
OpenEvals	0.2.0	MIT	★★★★	★★★★★ multimodal	-	Easy	Ter
SelfCheckGPT	0.1.7	MIT	★★★★ factuality	-	-	Easy	Ha
Lynx	v1.1	code Apache-2.0; weights CC-BY-NC-4.0	★★★★★	-	-	Easy	R& noi

Framework	Version	License	RAG	Agent	Coverage	Prom integ.	Ver
HHEM-2.1-Open	2.1	Apache-2.0 weights	★★★★	-	-	Easy	Pre hal

5. Agent-loop and self-optimization diagnostics

The agent loop's think/act/observe steps map cleanly to **OpenInference span kinds** (Apache-2.0, <https://github.com/Arize-ai/openinference>): the Soul cycle's planning/goal layer becomes a `CHAIN` root span with `gen_ai.agent.name=mindx.soul`; Mind reasoning is an `AGENT` span with child `LLM` spans for each think; Hands actions are `TOOL` spans (with `tool.name`, `tool.parameters`, `tool.output`) and `RETRIEVER` spans for memory queries; the self-eval pass is an `EVALUATOR` span with score attributes; input/output guardrails are `GUARDRAIL` spans. OpenInference span kinds are richer than the OTEL GenAI semconv (which currently defines only `create_agent` and `invoke_agent`) and are valid OTLP — Phoenix, Laminar, Langfuse, Arize, Jaeger v2 all ingest them. Plan migration to pure GenAI semconv when it stabilizes; for now, emit both via OpenLLMetry's instrumentations and let the Collector tag with `openinference.span.kind`, `OpenTelemetry`, `Agentmarketcap`

The recommended self-improvement metric battery, distilled from METR, Cambridge/ICML 2025 (van der Schaar group), the LMArena Goodhart paper (Cohere/Stanford/MIT 2025), and the Clune-group HyperAgents/ADAS/SAGE lineage: per-cycle delta on the held-out eval battery (`Δscore_t = score(checkpoint_t) - score(checkpoint_{t-1})`), emitted as `mindx_aglm_eval_delta{suite}`, reward/utility moving average over K cycles with stagnation and suspicious-jump alerts, partitioned tool-call success rate, retrieval hit rate (fraction of retrieve spans whose chunks are cited by the next think — falling ratio = retrieval drift), HHEM-sampled hallucination rate on production, plan-step efficiency via `tokens_used / task_completion` and `replan_count_per_task`, and METR's "50% reliability time horizon" (currently ~50 min for frontier, doubling every ~4 months). **Specification gaming defenses** are layered: held-out canary metrics that are never optimized against, distributional drift detection on the agent's action distribution (KL between cycle t and t-1), multi-metric agreement requiring ≥3 independent reward signals (LLM-judge, programmatic, retrieval-grounding, human spot-check) to all move up before claiming improvement, adversarial honeypot tasks where gaming-friendly shortcuts produce wrong answers, hold-out evaluator rotation (rotate judge model and prompt; gaming a single judge shows as judge-dependent variance), and trace-level invariants (e.g., "agent must never delete files outside `/workspace`") tracked even when tasks succeed. `o-mega` `Matt Hopkins`

Episode replay infrastructure: every agent loop emits an OTLP trace with full span attributes (input prompt, retrieved chunks, tool I/O); long-term store in SeaweedFS via Jaeger v2's S3 backend, keyed by `episode_id`/`trace_id`; replay tool re-instantiates the agent with mocked

tool outputs from the recorded trace. Open problems flagged: LLM stochasticity (mitigate by pinning model+seed+system fingerprint and recording full request/response), tool side-effects (record inputs *and* outputs; on replay mock tools by replaying recorded outputs, never re-executing), external-API drift (record HTTP-level request/response per tool call, VCR-style cassette), time/random (inject deterministic clock/PRNG seeds), RAG corpus drift (pin embedding-index snapshot by hash). **Phoenix's Span Replay** and **Laminar's agent rollout** are the two production tools in this space; both are OpenInference-compatible and can sit alongside Langfuse.

6. RAG and Knowledge Catalogue diagnostics

The Knowledge Catalogue's CQRS shape — append-only NATS JetStream (`KC_LOG`) projected by three independent consumers into Kuzu, Qdrant, and Meilisearch — has three diagnostic concerns: projection lag, cross-store consistency, and replay correctness.

Qdrant 1.17.x (Apache-2.0) exposes `/metrics` natively at `:6333` (per-node only, never via load-balancer URL), giving `rest_responses_avg_duration_seconds`, `grpc_responses_*`, `memory_resident_bytes`, `cluster_*`, `collections_total`, `collection_hardware_metric_*`. Per-collection breakdowns are still partial (issue #3322 open) — augment with the **community qdrant-exporter at :9153** for the 29-panel Qdrant Observatory dashboard with `indexed_vectors_ratio` (warns when search falls back to brute force — leading recall-degradation indicator), shard transfer progress, Raft health, and per-path-type latency. **Production recall@k pattern** (Qdrant has no built-in production recall metric since recall requires ground truth): for a 1-5% sample, run the production HNSW query and an `exact: true` reference query in parallel, compute `recall@k = |hits_hnsw n hits_exact| / k`, emit as `rag_recall_at_k{collection,k}` Histogram; pair with user-feedback recall when click-through is available. **RocksDB is removed in 1.17.x in favor of `gridstore`** — direct upgrades from 1.15.x are blocked, upgrade one minor at a time. `Qdrant + 2`

Meilisearch 1.38.2 (MIT) exposes `/metrics` at `:7700` but **the endpoint is still experimental after three years** (since v1.1 April 2023) — pin `MEILISEARCH_VERSION` in the image and re-validate dashboards on each upgrade; field-rename risk is real. Enable with `--experimental-enable-metrics`, scrape with a Bearer token whose API key has `metrics.get` and no index restrictions. Track `meilisearch_db_size_bytes`, `meilisearch_index_docs_count{index}`, `meilisearch_http_response_time_seconds_bucket` (the canonical query-latency histogram), `meilisearch_searches_running`, `meilisearch_searches_waiting_to_be_processed` (search-queue saturation, since v1.12+), `meilisearch_is_indexing`, `meilisearch_last_update`. Indexing-lag alert: `time() - meilisearch_last_update > 5m` paired with `meilisearch_is_indexing == 1 for >N`. `GitHub + 2`

Kuzu (MIT) is upstream archived as of October 10, 2025 (Kùzu Inc. announced a pivot; v0.11.3 is the final official release with `algo`, `fts`, `json`, `vector` extensions bundled inline). Choose between staying frozen on 0.11.3, migrating to the community fork **Bighorn** (Kineviz) or

Ladybug (Arun Sharma) — both MIT, both early-stage, both pledge ongoing maintenance — or evaluating an alternative embedded graph store. Pin a single fork. Kuzu has no native `/metrics` (it's an embedded library, no server process); ship a custom `kuzu_exporter.py` at `:9354/metrics` that wraps every `connection.execute()` in a `Histogram(name="kuzu_query_duration_seconds", labelnames=["query_class"])` with template-based classification (1-hop, k-hop, vector_index, fts), polls `CALL stats()` and `CALL show_buffer_manager_info()` for `kuzu_buffer_cache_hits_total`/`misses_total`, polls `CALL show_indexes()` for vector and FTS index sizes, emits `kuzu_db_size_bytes` from `os.stat` on the `.kz` file, and instruments the connection pool for in-flight gauges and queue-wait histograms. [GitHub](#) [GitHub](#)

NATS JetStream 2.12.x (Apache-2.0) is scraped via the official **prometheus-nats-exporter** at `:7777` (Apache-2.0, <https://github.com/nats-io/prometheus-nats-exporter>) invoked as `prometheus-nats-exporter -varz -connz -subz -routez -gatewayz -leafz -healthz -accstatz -jsz=all http://nats:8222`. The CQRS lag metrics are: `jetstream_consumer_num_pending{stream_name="KC_LOG", consumer_name="kc_proj_kuzu|qdrant|meili"}` (primary lag), `jetstream_consumer_num_redelivered` (poison-message indicator), `jetstream_consumer_delivered_consumer_seq` and `jetstream_stream_last_seq` (sequence progress). Helm-chart deployments override the default prefix to `nats` — use `grafana-jetstream-dash-helm.json` (Grafana dashboard ID **14725**) accordingly. **Note: NATS 2.12.5 had a regression around stream-update consumer loss in clustered deployments;** mitigate with `meta_compact_sync: true` until 2.12.6. [OpenObserve + 3](#)

The Knowledge Catalogue cross-store consistency probe runs every 60s: pick a random `record_id` from `KC_LOG` in the last 5-minute window and query each store. Emit `kc_consistency_probe_total{store,result="hit|miss|error"}`, `kc_consistency_divergence{a_store,b_store}` (0 if both hit/both miss, 1 if divergent), `kc_consistency_eventual_seconds{store}` Histogram of "time from JetStream publish to record-visible-in-store". For replay: `kc_replay_in_progress{store}`, `kc_replay_progress_ratio{store}`, `kc_replay_errors_total{store,reason}`, and at end of replay compute a content hash over each store's projection of the same key range — emit `kc_replay_hash_match{store}` (0/1); discrepancies indicate non-deterministic projection logic, a real CQRS bug class. Idempotency telemetry: `kc_projection_duplicate_skipped_total{store}` (healthy if non-zero — at-least-once redelivery is normal), `kc_projection_apply_failures_total{store,reason}`, `kc_projection_dlq_total` if a DLQ stream is implemented.

Embedding drift runs on a custom job every 15 minutes via **Evidently 0.7.21** (Apache-2.0, <https://github.com/evidentlyai/evidently>) — its `EmbeddingsDriftMetric` provides domain classifier (default ROC-AUC), MMD, share-of-drifted-components (per-component Wasserstein, threshold 0.1, drifted-share threshold 0.2), Euclidean/Cosine/Cityblock/Chebyshev between mean embeddings, and ratio variants, with PCA dim reduction. **Reject Alibi Detect** — relicensed

to **BUSL-1.1** in January 2024 (non-OSI, prohibits production use without paid Seldon license, Change License is Apache-2.0 only after 4 years per release). NannyML (Apache-2.0) is acceptable as a tabular post-deployment estimator; SciPy DIY (Wasserstein per-dimension, MMD via `sklearn.metrics.pairwise.rbf_kernel`, KL via histogram binning) is the cheap baseline. Emitted metrics: `mindx_embedding_drift_score{store,collection,method}`, `mindx_embedding_drift_share_drifted_components{collection}`, `mindx_embedding_reference_window_age_seconds{collection}`, `mindx_embedding_drift_alarm{collection}` 0/1 latch when score exceeds threshold.
 (Evidentlyai + 3)

7. Training pipeline diagnostics — automindXtrain v3.6

MLflow 3.12.0 (Apache-2.0, <https://github.com/mlflow/mlflow>) is the recommended tracking + registry layer. Reject ClearML (server is **SSPL-1.0**, non-OSI, hard fail on sovereignty). Reject W&B self-hosted (commercial, not OSS). Aim 3.29.1 (Apache-2.0) is a strong complement for high-cardinality scalar exploration but lacks a registry — use the `aimlflow` bridge if both are deployed. **DVC** (Apache-2.0, ownership migrating from `iterative/dvc` to `treeverse/dvc` after lakeFS acquisition November 2025; license unchanged, public OSS commitment) versions training datasets, tokenizer revisions, and base-model checkpoints; pin the remote URL in install scripts. The **dual-emit pattern** is mandatory because MLflow has no native `/metrics` endpoint: the training loop writes loss / lr / grad_norm / tokens_per_sec to MLflow (long-term) AND to `prometheus_client` Counters/Gauges/Histograms (short-term ops), so Grafana shows training-time metrics alongside production metrics under shared `aglm_release` and `tier` labels. DCGM exporter with the HPC job-mapping directory attributes every GPU sample to the running `automindx_run_id`, joinable with MLflow run IDs in PromQL. (GitHub + 2)

Checkpoint eval gating is the keystone. The legacy MLflow Stages API (`transition_model_version_stage`) is **deprecated since MLflow 2.9** — do not build new automation on it. The 2026 pattern uses **registered-model aliases + tags**: register every checkpoint passing a coarse threshold to the registry as a new version of `qwen3.6-automindx`, tag with `validation_status=pending|passing|failing`, `eval_run_id`, plus quality tags (`faithfulness=0.91`, `mmlu_pro=0.74`), assign `@candidate` automatically when the eval suite passes, flip `@champion` only after a multi-gate workflow checks (a) holdout loss < baseline, (b) RAG faithfulness ≥ floor, (c) safety/red-team checks, (d) latency budget on representative GPU, then promote via `MlflowClient.set_registered_model_alias(...)`. The CI workflow is GitHub Actions or a NATS-driven internal scheduler — "no green eval, no merge."

The DCGM training metric set, configured via custom counters CSV ConfigMap: utilization (`DCGM_FI_DEV_GPU_UTIL`, `DCGM_FI_PROF_SM_ACTIVE`, `DCGM_FI_PROF_SM_OCCUPANCY`) — better signal than `GPU_UTIL` for training, (`DCGM_FI_PROF_PIPE_TENSOR_ACTIVE`, `_FP16_ACTIVE`, `_FP32_ACTIVE`, `_FP64_ACTIVE`); VRAM (`DCGM_FI_DEV_FB_FREE/USED/TOTAL`); NVLink (`DCGM_FI_DEV_NVLINK_BANDWIDTH_TOTAL`, `DCGM_FI_PROF_NVLINK_TX_BYTES/RX_BYTES`),

`_CRC_FLIT_ERROR_COUNT_TOTAL`), `_REPLAY_ERROR_COUNT_TOTAL`); power & thermal (`DCGM_FI_DEV_POWER_USAGE`), `_GPU_TEMP`, `_MEMORY_TEMP`, `_POWER_VIOLATION`, `_THERMAL_VIOLATION`, `_BOARD_LIMIT_VIOLATION`); ECC (`DCGM_FI_DEV_ECC_SBE_VOL_TOTAL`, `_DBE_VOL_TOTAL`, `_ECC_SBE_AGG_TOTAL`, `_DBE_AGG_TOTAL`, `_UNCORRECTABLE_REMAPPED_ROWS`, `_ROW_REMAP_FAILURE`, `_XID_ERRORS`); clocks (`DCGM_FI_DEV_SM_CLOCK`, `_MEM_CLOCK`)).

DeepWiki + 3

8. openBDK toolset reuse

The openBDK observability assets are repackaged as a reusable `openbdk-observability` **toolset** that mindX consumes via Git submodule or vendored copy, structured around three shared artifacts: a **shared dashboard library** under `grafana/dashboards/shared/` (`cross_project_slo.json`, `infra_node_overview.json`, `tls_cert_health.json`, `alerting_overview.json`); a **shared alert rule library** under `prometheus/rules/shared_alert_rules.yml` covering blackbox probe down, TLS cert expiry <14 days, node disk fill, container OOMs, error budget burn rates per the Google SRE multi-window multi-burn-rate pattern; **shared recording rules** at `prometheus/rules/shared_recording_rules.yml` for the SLI bases (e.g., `service:probe_success:ratio_5m`, `service:request_duration_p95:5m`, `service:error_budget_burn:1h`). Both projects scrape with a unique `external_label` (`project=openbdk` vs `project=mindx`) and `cluster=` so the cross-project SLO dashboard can pivot without joins. The **cross-project SLO dashboard** is a single Grafana panel with rows per project, each row showing probe success, error budget burn rate at 1h/6h/24h windows, p95 latency vs SLO, and (mindX only) hallucination rate and per-tier eval-score sparklines — answering on one pane "is the openBDK bridge healthy AND is mindX's autonomous self-improvement actually working?"

The Grafana folder structure under one Org with RBAC roles `openbdk-{viewer,editor,admin}`, `mindx-{viewer,editor,admin}`, `shared-{viewer,editor}`:

```

/openbdk/                                (RBAC: openbdk-* roles)
  bridge_health.json
  parsec_wallet_metrics.json
/mindx/                                  (RBAC: mindx-* roles)
  api_overview.json
  agent_loop_traces.json
  aglm_outcome_quality.json
  knowledge_catalogue.json
  training_pipeline.json
  llm_observability.json
/shared/                                  (RBAC: shared-* roles, viewable by all)
  cross_project_slo.json
  infra_node_overview.json
  tls_cert_health.json
  alerting_overview.json

```

9. Deployment topology — Podman Quadlets, TLS, secrets, backup

Podman Quadlets (Podman 5.8, Apache-2.0, <https://github.com/containers/podman>) are the recommended production model for the single ops node — merged into Podman 4.4, mature in 5.x, systemd-native lifecycle, journald-integrated, dependency ordering via `Requires=`/`After=`, healthchecks, secrets, `podman auto-update`. Quadlets require cgroups v2 and the systemd cgroup manager (the default in modern distros). Quadlets are *transient* services regenerated on `daemon-reload`; rootless boot persistence requires `loginctl enable-linger`. Reject `podman-compose` for production (GPL-2.0, weakest systemd integration, fine for dev). `podman kube play` / `.kube` Quadlets are useful migration bridges only. (Red Hat + 3)

Rootless gotchas: Podman 5.0+ default network backend is `pasta` (passt project), not `slirp4netns`; rootless containers cannot bind ports <1024 unless `net.ipv4.ip_unprivileged_port_start=80` is set in `/etc/sysctl.d/99-rootless-ports.conf` (the `setcap cap_net_bind_service=+ep` workaround does not work reliably on Debian 13 / Podman 5.4.2 as of early 2026 per upstream regression). Image pulls may exceed the 90s startup default — set `TimeoutStartSec=900` explicitly. Pasta copies the host main-interface IP, so traffic from inside a container to that host IP is unreachable; use a custom `podman network create` for inter-container DNS. (Podman)

`podman_quadlets/otel_collector.container` — copy-pasteable:

```
ini
```

```
# ~/.config/containers/systemd/otel-collector.container
[Unit]
Description=OpenTelemetry Collector (mindX gateway)
After=network-online.target
Wants=network-online.target

[Container]
Image=docker.io/otel/opentelemetry-collector-contrib:0.151.0
ContainerName=otel-collector
AutoUpdate=registry
Network=obs.network
Volume=%h/obs/otel_collector_config.yaml:/etc/otelcol-contrib/config.yaml:Z,ro
Secret=langfuse_basic_auth,type=env,target=LANGFUSE_BASIC_AUTH
PublishPort=127.0.0.1:4317:4317
PublishPort=127.0.0.1:4318:4318
PublishPort=127.0.0.1:8888:8888
Exec--config=/etc/otelcol-contrib/config.yaml
HealthCmd=wget -q0- http://localhost:13133/ || exit 1
HealthInterval=30s
HealthRetries=3
Environment=GOMEMLIMIT=2GiB
Environment=OTEL_SEMCONV_STABILITY_OPT_IN=gen_ai_latest_experimental
LogDriver=journald
NoNewPrivileges=true
ReadOnly=true
Tmpfs=/tmp

[Service]
Restart=always
RestartSec=5
TimeoutStartSec=900

[Install]
WantedBy=default.target
```

TLS termination: Caddy v2 (Apache-2.0, automatic Let's Encrypt + ZeroSSL, HTTP/3 default, ~30-100MB resident) is the default; Traefik v3 (MIT) when service discovery + canary/mirror routing matters more (use Traefik for the mindX A/B + shadow deployment plane).

`prom.pythai.net`, `grafana.pythai.net`, `langfuse.pythai.net`, `jaeger.pythai.net` all sit behind a Caddyfile with `forward_auth` to **Keycloak 26.6.0** (Apache-2.0) or **Authentik 2026.2.2** (MIT core; flag that the same image bundles `/authentik/enterprise/` modules under

proprietary EE — pin to MIT-only paths or buy a license for production EE features). **Reject Zitadel** (relicensed to AGPL-3.0 in v3 2025). (DEV Community) (ZITADEL)

Secrets: sops + age (sops MPL-2.0 now CNCF Sandbox after Mozilla transfer, age BSD-3) for static config-as-code in Git, **OpenBao 2.5.0** (MPL-2.0, <https://github.com/openbao/openbao> — active Linux Foundation fork of Vault 1.14.0, API-compatible, the existing (hashicorp/vault) Terraform provider works unchanged, v2.5.0 added Namespaces and horizontal read scalability that were Vault-Enterprise-only) for dynamic secrets, PKI, and runtime injection. **Reject HashiCorp Vault** (relicensed to BUSL-1.1 August 2023). Podman secrets file driver is **not encrypted at rest** — base64-encoded JSON protected only by FS permissions; better than env vars in unit files but not a Vault replacement. (bespinian + 2)

Backup/restore: Prometheus snapshot via (--web.enable-admin-api) and (curl -XPOST http://prom:9090/api/v1/admin/tsdb/snapshot) (hard-linked blocks under (data/snapshots/)); Grafana = SQLite (/var/lib/grafana/grafana.db) or external Postgres + provisioning dirs + (grafana.ini) (with declarative dashboards in Git the DB is ephemeral); Loki = filesystem chunks + index OR object-store bucket; Jaeger v2 ClickHouse = standard (clickhouse-backup) to S3-compat. Backup tool: **Kopia** (Apache-2.0) preferred for sovereignty alignment, **Restic** (BSD-2) acceptable, both with chacha20-poly1305 encryption, S3/SFTP/B2/REST. (Robust Perception)

Scale-out path: Thanos sidecar already in place gives the long-term path. SeaweedFS (Apache-2.0) is the S3-compatible object store (~30k stars, mature, Go, billions-of-small-files specialty, Kubeflow Pipelines defaulted to it after MinIO entered maintenance mode). **MinIO is rejected** — repository entered maintenance mode December 2025, archived April 2026, Web UI moved to commercial AIStor (~\$96k/yr enterprise license). **Garage** (AGPL-3.0) and **RustFS** (Apache-2.0 but alpha) and **Ceph RGW** (LGPL-2.1, petabyte-scale, complex ops) are alternatives. **VictoriaMetrics** (Apache-2.0) is the drop-in single-binary alternative to Thanos when ops simplicity matters more than the sidecar ergonomics. (Elest + 5)

10. Concrete deliverable layout — (mindx_observability/)

The repository follows the (cypherpunk2048) flat snake_case convention, Apache 2.0 licensed:

```
mindx_observability/
├─ readme.md
├─ license (Apache 2.0)
├─ pyproject.toml (Python ≥ 3.12, snake_case modules)
├─ prometheus/
│   ├─ prometheus.yml
│   ├─ alertmanager.yml
│   └─ rules/
│       ├─ alert_rules.yml
│       ├─ recording_rules.yml
│       ├─ eval_drift_rules.yml (Goodhart canary, regression alerts)
│       ├─ shared_alert_rules.yml (openBDK + mindX shared)
│       └─ shared_recording_rules.yml
├─ otel_collector/
│   ├─ otel_collector_config.yaml (gateway profile)
│   └─ otel_agent_config.yaml (host agent profile)
├─ grafana/
│   ├─ grafana.ini
│   └─ provisioning/
│       ├─ datasources/
│       │   ├─ prometheus.yml
│       │   ├─ loki.yml
│       │   ├─ jaeger.yml
│       │   └─ thanos.yml
│       └─ langfuse.yml
│       ├─ dashboards/
│       │   ├─ dashboards.yml
│       │   ├─ openbdk/ (vendored from openbdk-observability)
│       │   └─ mindx/
│       │       ├─ api_overview.json
│       │       ├─ agent_loop_traces.json
│       │       ├─ aglm_outcome_quality.json
│       │       ├─ knowledge_catalogue.json
│       │       ├─ training_pipeline.json
│       │       └─ llm_observability.json
│       └─ shared/
│           ├─ cross_project_slo.json
│           ├─ infra_node_overview.json
│           ├─ tls_cert_health.json
│           └─ alerting_overview.json
│       └─ alerting/
│           └─ contact_points.yml
├─ podman_quadlets/
```

- | └─ obs.network
- | └─ prometheus.container
- | └─ alertmanager.container
- | └─ karma.container
- | └─ grafana.container
- | └─ thanos_sidecar.container
- | └─ thanos_querier.container
- | └─ otel_collector.container
- | └─ jaeger.container
- | └─ clickhouse.container
- | └─ loki.container
- | └─ vector.container
- | └─ seaweedfs.container
- | └─ caddy.container
- | └─ keycloak.container
- | └─ openbao.container
- | └─ pushgateway.container
- | └─ blackbox_exporter.container
- | └─ node_exporter.container
- | └─ dcgm_exporter.container
- | └─ prometheus_podman_exporter.container
- | └─ process_exporter.container
- | └─ nats_exporter.container
- | └─ qdrant_exporter.container
- | └─ kuzu_exporter.container
- | └─ meilisearch_exporter.container
- | └─ langfuse_web.container
- | └─ langfuse_worker.container
- | └─ langfuse_postgres.container
- | └─ langfuse_clickhouse.container
- | └─ langfuse_redis.container
- | └─ mlflow.container
- | └─ mlflow_postgres.container
- | └─ traefik.container (A/B + shadow router for
- mindx.pythai.net)
- | └─ exporters/
 - | └─ kuzu_exporter.py (custom Python exporter for Kuzu metrics)
 - | └─ meilisearch_metrics_proxy.py (token-bearing scrape proxy)
 - | └─ kc_consistency_probe.py (cross-store CQRS consistency probe)
 - | └─ kc_projection_lag_exporter.py (time-based lag, not just count)
 - | └─ ragas_live_sampler.py (NATS-consumer, live-traffic Ragas)
 - | └─ hhem_hallucination_sampler.py (HHEM-2.1-Open production sampler)
 - | └─ embedding_drift_exporter.py (Evidently-driven, scheduled)
 - | └─ shared/

- | └─ prometheus_helpers.py
- | └─ pushgateway_pusher.py
- └─ eval_runners/
 - | └─ aglm_eval_pusher.py (Pushgateway pattern, see §4)
 - | └─ lm_eval_runner.py (lm-evaluation-harness wrapper)
 - | └─ inspect_runner.py (Inspect AI wrapper + inspect_evals)
 - | └─ deepeval_runner.py
 - | └─ ragas_runner.py
 - | └─ promptfoo_ci_gate.sh
 - | └─ checkpoint_gate.py (MLflow alias-based promotion logic)
 - | └─ ab_shadow_orchestrator.py (Traefik weighted-routing controller)
 - | └─ batteries/
 - | └─ edge.yaml
 - | └─ mid.yaml
 - | └─ flagship.yaml
 - | └─ specialist_code.yaml
 - | └─ specialist_think.yaml
- └─ instrumentation/
 - | └─ fastapi_otel_setup.py (OTel + OpenLLMetry + Langfuse SDK v4)
 - | └─ agent_span_helpers.py (OpenInference span_kind helpers)
 - | └─ prometheus_metrics.py (custom mindX FastAPI metrics)
- └─ secrets/
 - | └─ secrets.enc.yaml (sops + age)
 - | └─ .sops.yaml
- └─ scripts/
 - | └─ bootstrap_node.sh (idempotent ops node setup)
 - | └─ deploy_quadlets.sh
 - | └─ backup_prometheus.sh
 - | └─ backup_grafana.sh
 - | └─ backup_clickhouse.sh
 - | └─ restore_prometheus.sh
 - | └─ rotate_certs.sh
 - | └─ upgrade_qdrant.sh (one-minor-at-a-time enforcer)
- └─ docs/
 - | └─ architecture.md
 - | └─ runbooks/
 - | └─ eval_regression.md
 - | └─ projection_lag.md
 - | └─ gpu_xid_error.md
 - | └─ shadow_promotion.md
 - | └─ license_inventory.md (SPDX inventory of every dependency)

`instrumentation/prometheus_metrics.py` — the custom mindX agent KPIs exposed at `/metrics`, copy-pasteable:

python

```
from prometheus_client import Counter, Histogram, Gauge, make_asgi_app
from fastapi import FastAPI

agent_actions_total = Counter(
    'mindx_agent_actions_total', 'Total agent actions by outcome',
    ['agent', 'action_type', 'tool', 'outcome', 'aglm_release', 'tier'])
agent_action_duration_seconds = Histogram(
    'mindx_agent_action_duration_seconds', 'Agent action duration',
    ['agent', 'action_type', 'tool'],
    buckets=(0.05, 0.1, 0.25, 0.5, 1, 2, 5, 10, 30, 60, 120, 300))
tool_call_success_ratio_window = Gauge(
    'mindx_tool_call_success_ratio_5m', 'Rolling 5m tool-call success ratio',
    ['tool', 'aglm_release'])
retrieval_hit_rate = Histogram(
    'mindx_retrieval_hit_rate', 'Fraction of retrieved chunks cited by next think',
    ['retriever', 'aglm_release'],
    buckets=(0.1, 0.25, 0.5, 0.75, 0.9, 1.0))
hallucination_score = Histogram(
    'mindx_hallucination_score', 'HHEM-2.1-Open hallucination score (0..1)',
    ['aglm_release', 'tier'],
    buckets=(0.0, 0.1, 0.2, 0.3, 0.5, 0.7, 0.9, 1.0))
plan_replan_count = Histogram(
    'mindx_plan_replan_count', 'Replans per task',
    ['agent', 'aglm_release'],
    buckets=(0, 1, 2, 3, 5, 8, 13, 21))
task_completion_total = Counter(
    'mindx_task_completion_total', 'Task completion outcomes',
    ['outcome', 'aglm_release', 'tier']) # outcome: strict|lenient|audited|failed
canary_invariant_violations_total = Counter(
    'mindx_canary_invariant_violations_total',
    'Trace-level invariant violations (Goodhart sentinel)',
    ['invariant', 'aglm_release'])

def attach(app: FastAPI):
    app.mount('/metrics', make_asgi_app())
```

11. Final stack and license inventory

The recommended stack, all Apache-2.0 / MIT / MPL-2.0 / BSD where possible, with AGPL-3.0 components used unmodified at network boundaries:

Excluded (license/maintenance flags): ClearML server (SSPL-1.0); Vault (BUSL-1.1) → use OpenBao; Alibi Detect (BUSL-1.1) → use Evidently; MinIO (AGPL-3.0 + maintenance mode) → use SeaweedFS; Lunary (repo deleted Dec 2025); Helicone (Mintlify maintenance mode); Mimir/Tempo/Loki/Alloy (AGPL-3.0 — Tempo/Alloy replaced by Jaeger v2 / Vector; Loki

accepted at arm's length); Zitadel (relicensed to AGPL-3.0); Patronus Lynx weights (CC-BY-NC-4.0) → use HHEM-2.1-Open; CRAG dataset (CC-BY-NC-4.0) → use RGB + MultiHop-RAG; Promtail (EOL March 2 2026); cAdvisor for Podman → use prometheus-podman-exporter; original Kuzu upstream (archived Oct 10 2025) → pin 0.11.3 or migrate to Bighorn/Ladybug fork.

12. Industry-standard alignment

All custom mindX instrumentation conforms to **OpenTelemetry GenAI Semantic Conventions** (<https://opentelemetry.io/docs/specs/semconv/gen-ai/>, status Development as of May 2026 — pin v1.36.0 baseline, opt into v1.40 via

`(OTEL_SEMCONV_STABILITY_OPT_IN=gen_ai_latest_experimental)`); use `(gen_ai.provider.name)` not the deprecated `(gen_ai.system)`; capture full prompt/completion content as **events** `((gen_ai.client.inference.operation.details))` gated by `(OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT)`, never as span attributes; emit `(gen_ai.client.operation.duration)` histogram, `(gen_ai.client.token.usage)` histogram with `(gen_ai.token.type=input|output|reasoning)`, and the new `(gen_ai.evaluation.result)` event for eval scores. The **SIG GenAI Observability** working group (active since April 2024 under the Semantic Conventions SIG, weekly meetings, contributors from Microsoft, Elastic, Google, IBM, Traceloop, Langtrace, Scorecard, OpenLIT, Amazon, Anthropic MCP integration in progress) drives the spec; mindX should track its monthly state and migrate when Stable is declared. **OpenInference** (Arize) is the practical superset used in production today and will harmonize with stable GenAI semconv when published. `(OpenTelemetry)`

The CNCF observability landscape — Kubernetes (Graduated 2018), Prometheus (Graduated 2018), OpenTelemetry (Incubating, second-highest velocity in CNCF after K8s), Jaeger (Graduated Oct 2019), Fluentd + Fluent Bit (Graduated 2019), Cortex (Incubating), Thanos (Incubating), OpenCost (Incubating Oct 2024), Pixie (Sandbox), Falco (Graduated 2024) — defines the gravitational center of the stack. Grafana Labs is a CNCF member but Grafana itself is not a CNCF project (and is AGPL-3.0 since 2021, as are Loki, Tempo, Mimir, Alloy) — this is the only point where the mindX stack departs from CNCF/Apache-2.0 purity, mitigated by using these components unmodified at process boundaries with no derivative-works exposure. CNCF launched a dedicated **Open Observability Summit** in 2025; mindX should track talks there for emerging best practices.

Conclusion — what changes when this is deployed

The autonomous self-improvement claim becomes empirically falsifiable. Every aGLM checkpoint promotion is gated by a multi-metric, contamination-aware, statistically rigorous eval battery whose results live in the same Prometheus TSDB as latency, GPU utilization, and Knowledge Catalogue projection lag — meaning regression, drift, and Goodhart-gaming all become PromQL queries and Alertmanager rules rather than ad-hoc reports. The cross-project

SLO dashboard reduces "is mindX healthy AND is openBDK healthy" to one pane while sovereign data ownership is preserved by Thanos sidecar federation rather than commingled TSDBs. The tracing layer captures the full Soul-Mind-Hands cognitive flow as OpenInference spans, replayable from object storage, with PII redaction at the Collector boundary and content capture gated by env var. The LLM observability backend (Langfuse v3 MIT) provides per-checkpoint regression analytics, prompt versioning, and A/B dataset comparisons that compose with — rather than duplicate — the Prometheus eval metrics. The deployment plane is Podman Quadlets with `auto-update`, sops+age secrets, OpenBao for dynamic injection, Caddy auto-HTTPS, OIDC via Keycloak, and SeaweedFS object storage — every component Apache-2.0 / MIT / MPL-2.0 / BSD with three explicit AGPL-3.0 acceptances (Grafana, Loki — used unmodified) and one Elastic-2.0 acceptance (Phoenix as developer dev tool). The novel insight worth carrying forward: **outcome quality is the highest-value telemetry signal in an autonomous self-improving system**, and treating it as a first-class Prometheus metric — keyed by `aglm_release`, `tier`, `checkpoint_id`, with held-out canaries and divergence monitors — turns "is the autonomous loop actually working?" from a vibes question into a single dashboard panel and a paged alert. The blueprint above is copy-pasteable; clone the layout, fill the blanks, ship it.